Q: An e-commerce platform integrates multiple payment gateways to process customer transactions. The platform expects all payment gateways to provide a **processPayment** method to handle payments. However, one of the newly added gateways implements a different method, **make_transaction**. You have been tasked with modifying the system to ensure that the new gateway works seamlessly without changing the e-commerce platform's code.

1. Considering the above scenario, choose an appropriate design pattern. Justify your answer.
2. Write the code necessary to implement the modifications required by the e-commerce platform.
3. What type of design pattern is the adapter pattern? Explain your answer.

Driver Code:

```
class PaymentGateway:
        def process_payment(self, amount):
            raise NotImplementedError("Subclasses must implement this method.")

class BankGateway(PaymentGateway):
        def process_payment(self, amount):
            print(f"Processing payment of ${amount} through Bank Gateway.")

class ThirdPartyPay:
        def make_transaction(self, value):
            print(f"Processing payment of ${value} through Third Party Pay Gateway.")

class PaymentProcessor:
        def process_payment(self, gateway, amount):
            gateway.process_payment(amount)

if __name__ == "__main__":
        ecommerce_platform = PaymentProcessor()

        bank_gateway = BankGateway()
        ecommerce_platform.process_payment(bank_gateway, 100)

        third_party_pay = ThirdPartyPay()

        # The following line will currently raise an error
        ecommerce_platform.process_payment(third_party_pay, 100)
```

Answer:

1. [Questions like this must focus on the intent and motivation of the chosen design pattern. You must show how the scenario aligns with the motivations of the design pattern, and how the intent of the design pattern will solve the issue.]

The Adapter Pattern is suitable for this scenario. It allows the ThirdPartyPay class to be adapted to the interface expected by the PaymentProcessor class without modifying the existing e-commerce platform code or the ThirdPartyPay class. This ensures seamless integration of the new payment gateway.

The scenario has a client, the PaymentProcessor, expecting a particular target interface which is the PaymentGateway. But a third party class, the ThirdPartyPay has a different interface which our client must also use. This aligns with the motivations of the adapter pattern, and by implementing it, we would be able to convert the interface of the ThirdPartyPay class into the PaymentGateway interface that the client expects. Adapter lets classes work together that couldn't otherwise because of incompatible interfaces.
So, I would choose the Adapter design pattern for the above scenario.

2.

```
class PaymentGateway:
        def process_payment(self, amount):
            raise NotImplementedError("Subclasses must implement this method.")

class BankGateway(PaymentGateway):
        def process_payment(self, amount):
            print(f"Processing payment of ${amount} through Bank Gateway.")

class ThirdPartyPay:
        def make_transaction(self, value):
            print(f"Processing payment of ${value} through Third Party Pay Gateway.")

class PaymentProcessor:
        def process_payment(self, gateway, amount):
            gateway.process_payment(amount)

# This is an Object adapter that takes the Object as a parameter
# This allows the most compatibility with the driver code, since the Object exists already
# Normal Class adapters or Object adapters are also accepted answers
class ThirdPartyPayAdapter(PaymentGateway):
        def __init__(self, third_party_pay):
                self.third_party_pay = third_party_pay

        def process_payment(self, amount):
                self.third_party_pay.make_transaction(amount)
# This is an Class adapter
class ThirdPartyPayAdapter(PaymentGateway, ThirdPartyPay):
        def process_payment(self, amount):
                this.make_transaction(amount)

if __name__ == "__main__":
        ecommerce_platform = PaymentProcessor()

        bank_gateway = BankGateway()
        ecommerce_platform.process_payment(bank_gateway, 100)
```

```
third_party_pay = ThirdPartyPay()
adapter = ThirdPartyPayAdapter(third_party_pay)
ecommerce_platform.process_payment(adapter, 100)
```

3. The Adapter Pattern is classified as a structural design pattern because it focuses on how objects and classes are composed to form larger structures.
Specifically, it modifies structure without changing behavior. The pattern changes how the system is organized by introducing an intermediary (the adapter class) that reshapes the interface of one class to match another. This does not alter the underlying logic or behavior of the existing classes. It emphasizes the composition of objects to achieve functionality rather than modifying or inheriting from the original class.

Q: A chat application provides group chat functionality. When a message is sent to a group, all users in the group should be notified in real-time. Users can join or leave the group at any time, and the system must handle subscriptions dynamically.

1. Considering the above scenario, choose an appropriate design pattern. Justify your answer.
2. Write the code necessary to complete the provided driver code by implementing your chosen design pattern.
3. What are the different types of Design Patterns? Please provide examples for each type.

Driver Code:

```
if __name__ == "__main__":
        # Central chat group
        chat_group = ChatGroup("Developers")

        # Create users
        alice = User("Alice")
        bob = User("Bob")
        charlie = User("Charlie")

        # Subscribe users to the chat group
        chat_group.subscribe(alice)
        chat_group.subscribe(bob)

        # Send a message
        chat_group.send_message("Welcome to the Developers group!")
        # Notifies Alice and Bob

        # Charlie joins the group
        chat_group.subscribe(charlie)

        # Send another message
        chat_group.send_message("Charlie has joined!")
        # Notifies Alice, Bob, and Charlie

        # Bob leaves the group
        chat_group.unsubscribe(bob)

        # Send one more message
        chat_group.send_message("Goodbye Bob!")  # Notifies Alice and Charlie
```

Answer:

1. The **Observer Pattern** is suitable for this scenario. It ensures that when a message is sent to the group (the subject), all subscribed users (observers) are notified in real-time. The Observer Pattern dynamically manages the subscriptions, making it easy to handle users joining or leaving the group.

In this scenario, many users are dependent on the state of the central chat group in order to communicate with each other, which aligns with the Observer pattern. It would allow us to define this one to many dependency so that when the state of the chat group changes, all the users would be notified and get real time updates.
So, I would choose the Observer pattern for the above scenario.
2.
[Remember to **strictly** maintain the names of the functions used in the driver code. In the driver code, the functions are called subscribe, unsubscribe and send_message. Do not forcibly change the driver code in order to match the standard structure. Also since send_message is used instead of set_state, you can skip get_state and modify the Observer's state directly. You can do these modifications as you see fit, as long as you maintain the core functionality of the observer pattern]

```
class ChatGroup:
        def __init__(self, name):
        self.name = name
        self.subscribers = []
        self.messages = []

        def subscribe(self, user):
        self.subscribers.append(user)

        def unsubscribe(self, user):
        self.subscribers.remove(user)

        def send_message(self, message):
        self.messages.append(message)
        for subscriber in self.subscribers:
        subscriber.notify(self.name, message)


class User:
        def __init__(self, name):
        self.name = name
        self.messages = []

        def notify(self, group_name, message):
        self.messages.append(message)
        print(f"{self.name} received message from {group_name}: {message}")

if __name__ == "__main__":
        # Central chat group
        chat_group = ChatGroup("Developers")
```

```
# Create users
alice = User("Alice")
bob = User("Bob")
charlie = User("Charlie")

# Subscribe users to the chat group
chat_group.subscribe(alice)
chat_group.subscribe(bob)

# Send a message
chat_group.send_message("Welcome to the Developers group!")
# Notifies Alice and Bob

# Charlie joins the group
chat_group.subscribe(charlie)

# Send another message
chat_group.send_message("Charlie has joined!")
# Notifies Alice, Bob, and Charlie

# Bob leaves the group
chat_group.unsubscribe(bob)

# Send one more message
chat_group.send_message("Goodbye Bob!")  # Notifies Alice and Charlie
```

Q: A weather monitoring application collects weather data (temperature, humidity, wind speed) from sensors deployed in various locations. A central system processes this data and notifies subscribed clients (such as mobile apps, websites, and news agencies) about updates in real-time. The central system must ensure a single instance to maintain consistent data.

1. Considering the above scenario, choose an appropriate design pattern. Justify your answer.
2. Write the code necessary to complete the provided driver code by implementing your chosen design pattern.
3. How would you differentiate between a behavioral pattern and a structural pattern? Explain your answer.

```
if __name__ == "__main__":
        # Central weather system
        weather_system = WeatherSystem()

        # Create clients
        app_client = WeatherClient("Mobile App")
        website_client = WeatherClient("Weather Website")

        # Subscribe clients to weather updates
        weather_system.subscribe(app_client)
        weather_system.subscribe(website_client)

        # Update weather data
        weather_system.update_weather_data(temperature=25, humidity=60,
wind_speed=10)  # Notifies both clients

        # Add another client
        news_agency = WeatherClient("News Agency")
        weather_system.subscribe(news_agency)

        # Update weather data again
        weather_system.update_weather_data(temperature=28, humidity=55,
wind_speed=12)  # Notifies all three clients
```

Answer:

1. This scenario requires combining two design patterns:

   a. **Singleton Pattern**: Ensures the central weather system has a single instance to maintain consistent data across the system.
   b. **Observer Pattern**: Enables clients to subscribe to the central system for real-time updates about weather changes.

2.

```
class WeatherSystem:
        _instance = None

        def __new__(cls, *args, **kwargs):
        if not cls._instance:
        cls._instance = super(WeatherSystem, cls).__new__(cls)
        cls._instance.subscribers = []
        cls._instance.weather_data = {}
        return cls._instance

        def subscribe(self, client):
        self.subscribers.append(client)

        def unsubscribe(self, client):
        self.subscribers.remove(client)

        def update_weather_data(self, temperature, humidity, wind_speed):
        self.weather_data = {
        "temperature": temperature,
        "humidity": humidity,
        "wind_speed": wind_speed,
        }
        self.notify_clients()

        def notify_clients(self):
        for subscriber in self.subscribers:
        subscriber.update(self.weather_data)


class WeatherClient:
        def __init__(self, name):
        self.name = name

        def update(self, weather_data):
        print(f"{self.name} received weather update: {weather_data}")


if __name__ == "__main__":
        # Central weather system
        weather_system = WeatherSystem()

        # Create clients
        app_client = WeatherClient("Mobile App")
```

```
website_client = WeatherClient("Weather Website")

# Subscribe clients to weather updates
weather_system.subscribe(app_client)
weather_system.subscribe(website_client)

# Update weather data
weather_system.update_weather_data(temperature=25, humidity=60, wind_speed=10)

# Add another client
news_agency = WeatherClient("News Agency")
weather_system.subscribe(news_agency)

# Update weather data again
weather_system.update_weather_data(temperature=28, humidity=55, wind_speed=12)
```